

Parte I

El Proceso Unificado de Desarrollo de Software

En esta Parte I presentamos las ideas fundamentales.

El Capítulo 1 describe el Proceso Unificado de Desarrollo de Software brevemente, centrándose en su carácter dirigido por los casos de uso, centrado en la arquitectura, iterativo e incremental. El proceso utiliza el Lenguaje Unificado de Modelado (UML), un lenguaje que produce dibujos comparables en sus objetivos a los esquemas que se utilizan desde hace mucho tiempo en otras disciplinas técnicas. El proceso pone en práctica el basar gran parte del proyecto de desarrollo en componentes reutilizables, es decir, en piezas de software con una interfaz bien definida.

El Capítulo 2 introduce las cuatro “P”: personas, proyecto, producto, y proceso, y describe sus relaciones, que son esenciales para la comprensión del resto del libro. Los conceptos clave necesarios para comprender el proceso que cubre este libro son: *artefacto*, *modelo*, *trabajador* y *flujo de trabajo*.

El Capítulo 3 trata el concepto de desarrollo dirigido por casos de uso con mayor detalle. Los casos de uso son un medio para determinar los requisitos correctos y utilizarlos para conducir el proceso de desarrollo.

El Capítulo 4 describe el papel de la arquitectura en el Proceso Unificado. La arquitectura establece lo que se tiene que hacer; esquematiza los niveles significativos de la organización del software y se centra en el armazón del sistema.

El Capítulo 5 enfatiza la importancia de adoptar una aproximación *iterativa e incremental* en el desarrollo de software. En la práctica, esto se traduce en atacar primero las partes del sistema

cargadas de riesgo, obteniendo pronto una arquitectura estable, y completando después las partes más rutinarias en iteraciones sucesivas, cada una de las cuales lleva a un incremento del progreso hasta la versión final.

En la Parte II profundizaremos más. Dedicamos un capítulo a cada flujo de trabajo fundamental: requisitos, análisis, diseño, implementación y prueba. Estos flujos de trabajo se utilizarán después en la Parte III como actividades importantes en los diferentes tipos de iteración durante las cuatro fases en las que dividimos el proceso.

En la Parte III describimos en concreto cómo se lleva a cabo el trabajo en cada fase: en la de inicio para obtener un análisis del negocio, en la de elaboración para crear la arquitectura y hacer un plan, en la de construcción para aumentar la arquitectura hasta conseguir un sistema entregable, y en la de transición para garantizar que el sistema funciona correctamente en el entorno del usuario. En esta parte, volvemos a utilizar los flujos de trabajo fundamentales y los combinamos de un modo ajustado a cada fase de manera que seamos capaces de alcanzar los resultados deseados.

La intención subyacente de una organización no es, sin embargo, tener un software bueno, sino administrar sus procesos de negocio, o sistemas empujados, de forma que le permita producir rápidamente bienes y servicios de alta calidad con costes razonables, en respuesta a las demandas del mercado. El software es el arma estratégica con el cual las empresas o los gobiernos pueden conseguir enormes reducciones en costes y tiempos de producción tanto para bienes como para servicios. Es imposible reaccionar con rapidez frente al dinamismo del mercado sin unos buenos procesos de organización establecidos. En el entorno de una economía global que opera veinticuatro horas al día, siete días a la semana, muchos de esos procesos no pueden funcionar sin el software. Un buen proceso de desarrollo de software es, por tanto, un elemento crítico para el éxito de cualquier organización.

Capítulo 1

El Proceso Unificado: dirigido por casos de uso, centrado en la arquitectura, iterativo e incremental

La tendencia actual en el software lleva a la construcción de sistemas más grandes y más complejos. Esto es debido en parte al hecho de que los computadores son más potentes cada año, y los usuarios, por tanto, esperan más de ellos. Esta tendencia también se ha visto afectada por el uso creciente de Internet para el intercambio de todo tipo de información —de texto sin formato a texto con formato, fotos, diagramas y multimedia. Nuestro apetito de software aún más sofisticado crece a medida que vemos cómo pueden mejorarse los productos de una versión a otra. Queremos un software que esté mejor adaptado a nuestras necesidades, pero esto, a su vez, simplemente hace el software más complejo. En breve, queremos más.

También lo queremos más rápido. El tiempo de salida al mercado es otro conductor importante.

Conseguirlo, sin embargo, es difícil. Nuestra demanda de software potente y complejo no se corresponde con cómo se desarrolla el software. Hoy, la mayoría de la gente desarrolla software mediante los mismos métodos que llevan utilizándose desde hace 25 años. Esto es un problema. A menos que renovemos nuestros métodos, no podremos cumplir con el objetivo de desarrollar el software complejo que se necesita actualmente.

El problema del software se reduce a la dificultad que afrontan los desarrolladores para coordinar las múltiples cadenas de trabajo de un gran proyecto de software. La comunidad de desarrolladores necesita una forma coordinada de trabajar. Necesita un proceso que integre las múltiples facetas del desarrollo. Necesita un método común, un proceso que:

- Proporcione una guía para ordenar las actividades de un equipo.
- Dirija las tareas de cada desarrollador por separado y del equipo como un todo.
- Especifique los artefactos que deben desarrollarse.
- Ofrezca criterios para el control y la medición de los productos y actividades del proyecto.

La presencia de un proceso bien definido y bien gestionado es una diferencia esencial entre proyectos hiperproductivos y otros que fracasan. (Véase la Sección 2.4.4 para más motivos por los cuales es necesario un proceso.) El Proceso Unificado de Desarrollo —el resultado de más de 30 años de experiencia— es una solución al problema del software. Este capítulo proporciona una visión general del Proceso Unificado completo. En posteriores capítulos, examinaremos cada elemento del proceso en detalle.

1.1. El Proceso Unificado en pocas palabras

En primer lugar, el Proceso Unificado es un proceso de desarrollo de software. Un *proceso* de desarrollo de software es el conjunto de actividades necesarias para transformar los requisitos de un usuario en un sistema software (véase la Figura 1.1). Sin embargo, el Proceso Unificado es más que un simple proceso; es un marco de trabajo genérico que puede especializarse para una gran variedad de sistemas software, para diferentes áreas de aplicación, diferentes tipos de organizaciones, diferentes niveles de aptitud y diferentes tamaños de proyecto.

El Proceso Unificado está *basado en componentes*, lo cual quiere decir que el sistema software en construcción está formado por **componentes** software (Apéndice A) interconectados a través de **interfaces** (Apéndice A) bien definidas.

El Proceso Unificado utiliza el *Lenguaje Unificado de Modelado* (Unified Modeling Language, UML) para preparar todos los esquemas de un sistema software. De hecho, UML es una parte esencial del Proceso Unificado —sus desarrollos fueron paralelos.

No obstante, los verdaderos aspectos definitorios del Proceso Unificado se resumen en tres frases clave —dirigido por casos de uso, centrado en la arquitectura, e iterativo e incremental. Esto es lo que hace único al Proceso Unificado.

En las tres secciones siguientes describiremos esas tres frases clave. Después, en el resto del capítulo, daremos una breve visión general del proceso: su ciclo de vida, fases, versiones, iteraciones, flujos de trabajo, y artefactos. Toda la motivación de este capítulo es la de presentar las ideas más importantes y dar una “vista de pájaro” del proceso en su totalidad. Después de este capítulo, el lector debería conocer, pero no necesariamente comprender en su totalidad, de qué trata el Proceso Unificado. El resto del libro cubrirá los detalles. En el Capítulo 2 establecemos el contexto de las cuatro “P” del desarrollo de software: personas, proyecto, producto y proceso. Después, dedicamos un capítulo a cada una de las tres ideas clave antes mencionadas. Todo esto constituirá la primera parte del libro. Las Partes II y III —el grueso del libro— describirán los distintos flujos de trabajo del proceso en detalle.

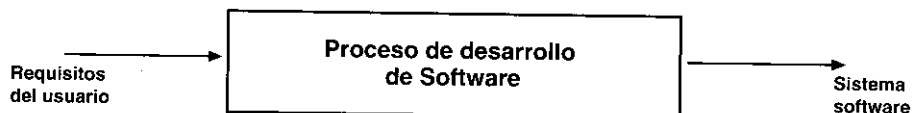


Figura 1.1. Un proceso de desarrollo de software.

1.2. El Proceso Unificado está dirigido por casos de uso

Un sistema software ve la luz para dar servicio a sus usuarios. Por tanto, para construir un sistema con éxito debemos conocer lo que sus futuros usuarios necesitan y desean.

El término *usuario* no sólo hace referencia a usuarios humanos sino a otros sistemas. En este sentido, el término *usuario* representa alguien o algo (como otro sistema fuera del sistema en consideración) que interactúa con el sistema que estamos desarrollando. Un ejemplo de interacción sería una persona que utiliza un cajero automático. Él (o ella) inserta la tarjeta de plástico, responde a las preguntas que le hace la máquina en su pantalla, y recibe una suma de dinero. En respuesta a la tarjeta del usuario y a sus contestaciones, el sistema lleva a cabo una secuencia de **acciones** (Apéndice A) que proporcionan al usuario un resultado importante, en este caso, la retirada del efectivo.

Una interacción de este tipo es un **caso de uso** (Apéndice A; véase también el Capítulo 3). Un caso de uso es un fragmento de funcionalidad del sistema que proporciona al usuario un resultado importante. Los casos de uso representan los requisitos funcionales. Todos los casos de uso juntos constituyen el **modelo de casos de uso** (Apéndice B; véase también la Sección 2.3), el cual describe la funcionalidad total del sistema. Puede decirse que una especificación funcional contesta a la pregunta: ¿Qué debe hacer el sistema?. La estrategia de los casos de uso puede describirse añadiendo tres palabras al final de esta pregunta: *¿...para cada usuario?* Estas tres palabras albergan una implicación importante. Nos fuerzan a pensar en términos de importancia para el usuario y no sólo en términos de funciones que sería bueno tener. Sin embargo, los casos de uso no son sólo una herramienta para especificar los requisitos de un sistema. También guían su diseño, implementación, y prueba; esto es, *guían el proceso de desarrollo*. Basándose en el modelo de casos de uso, los desarrolladores crean una serie de modelos de diseño e implementación que llevan a cabo los casos de uso. Los desarrolladores revisan cada uno de los sucesivos modelos para que sean conformes al modelo de casos de uso. Los ingenieros de prueba prueban la implementación para garantizar que los componentes del modelo de implementación implementan correctamente los casos de uso. De este modo, los casos de uso no sólo inician el proceso de desarrollo sino que le proporcionan un hilo conductor. *Dirigido por casos de uso* quiere decir que el proceso de desarrollo sigue un hilo —avanza a través de una serie de flujos de trabajo que parten de los casos de uso. Los casos de uso se especifican, se diseñan, y los casos de uso finales son la fuente a partir de la cual los ingenieros de prueba construyen sus casos de prueba.

Aunque es cierto que los casos de uso guían el proceso, no se desarrollan aisladamente. Se desarrollan a la vez que la arquitectura del sistema. Es decir, los casos de uso guían la arquitectura del sistema y la arquitectura del sistema influye en la selección de los casos de uso. Por tanto, tanto la arquitectura del sistema como los casos de uso maduran según avanza el ciclo de desarrollo.

1.3. El Proceso Unificado está centrado en la arquitectura

El papel de la arquitectura software es parecido al papel que juega la arquitectura en la construcción de edificios. El edificio se contempla desde varios puntos de vista: estructura, servicios, conducción de la calefacción, fontanería, electricidad, etc. Esto permite a un constructor ver una imagen completa antes de que comience la construcción. Análogamente, la arquitectura en un sistema software se describe mediante diferentes vistas del sistema en construcción.

El concepto de arquitectura software incluye los aspectos estáticos y dinámicos más significativos del sistema. La arquitectura surge de las necesidades de la empresa, como las perciben los usuarios y los inversores, y se refleja en los casos de uso. Sin embargo, también se ve influida por muchos otros factores, como la plataforma en la que tiene que funcionar el software (arquitectura hardware, sistema operativo, sistema de gestión de base de datos, protocolos para comunicaciones en red), los bloques de construcción reutilizables de que se dispone (por ejemplo, un **marco de trabajo** (Apéndice C) para interfaces gráficas de usuario), consideraciones de implantación, sistemas heredados, y requisitos no funcionales (por ejemplo, rendimiento, fiabilidad). La arquitectura es una vista del diseño completo con las características más importantes resaltadas, dejando los detalles de lado. Debido a que lo que es significativo depende en parte de una valoración, que a su vez, se adquiere con la experiencia, el valor de una arquitectura depende de las personas que se hayan responsabilizado de su creación. No obstante, el proceso ayuda al arquitecto a centrarse en los objetivos adecuados, como la comprensibilidad, la capacidad de adaptación al cambio, y la reutilización.

¿Cómo se relacionan los casos de uso y la arquitectura? Cada producto tiene tanto una función como una forma. Ninguna es suficiente por sí misma. Estas dos fuerzas deben equilibrarse para obtener un producto con éxito. En esta situación, la función corresponde a los casos de uso y la forma a la arquitectura. Debe haber interacción entre los casos de uso y la arquitectura. Es un problema del tipo “el huevo y la gallina”. Por un lado, los casos de uso deben encajar en la arquitectura cuando se llevan a cabo. Por otro lado, la arquitectura debe permitir el desarrollo de todos los casos de uso requeridos, ahora y en el futuro. En realidad, tanto la arquitectura como los casos de uso deben evolucionar en paralelo.

Por tanto, los arquitectos moldean el sistema para darle una *forma*. Es esta forma, la arquitectura, la que debe diseñarse para permitir que el sistema evolucione, no sólo en su desarrollo inicial, sino también a lo largo de las futuras generaciones. Para encontrar esa forma, los arquitectos deben trabajar sobre la comprensión general de las funciones clave, es decir, sobre los casos de uso claves del sistema. Estos casos de uso clave pueden suponer solamente entre el 5 y el 10 por ciento de todos los casos de uso, pero son los significativos, los que constituyen las funciones fundamentales del sistema. De manera resumida, podemos decir que el arquitecto:

- Crea un esquema en borrador de la arquitectura, comenzando por la parte de la arquitectura que no es específica de los casos de uso (por ejemplo, la plataforma). Aunque esta parte de la arquitectura es independiente de los casos de uso, el arquitecto debe poseer una comprensión general de los casos de uso antes de comenzar la creación del esquema arquitectónico.
- A continuación, el arquitecto trabaja con un subconjunto de los casos de uso especificados, con aquellos que representen las funciones clave del sistema en desarrollo. Cada caso de uso seleccionado se especifica en detalle y se realiza en términos de **subsistemas** (Apéndice A; véase también Sección 3.4.4), **clases** (Apéndice A), y **componentes** (Apéndice A).
- A medida que los casos de uso se especifican y maduran, se descubre más de la arquitectura. Esto, a su vez, lleva a la maduración de más casos de uso.

Este proceso continúa hasta que se considere que la arquitectura es estable.

1.4. El Proceso Unificado es iterativo e incremental

El desarrollo de un producto software comercial supone un gran esfuerzo que puede durar entre varios meses hasta posiblemente un año o más. Es práctico dividir el trabajo en partes más

pequeñas o miniproyectos. Cada miniproyecto es una iteración que resulta en un incremento. Las iteraciones hacen referencia a pasos en el flujo de trabajo, y los incrementos, al crecimiento del producto. Para una efectividad máxima, las iteraciones deben estar *controladas*; esto es, deben seleccionarse y ejecutarse de una forma planificada. Es por esto por lo que son *mini-proyectos*.

Los desarrolladores basan la selección de lo que se implementará en una iteración en dos factores. En primer lugar, la iteración trata un grupo de casos de uso que juntos amplían la utilidad del producto desarrollado hasta ahora. En segundo lugar, la iteración trata los riesgos más importantes. Las iteraciones sucesivas se construyen sobre los artefactos de desarrollo tal como quedaron al final de la última iteración. Al ser miniproyectos, comienzan con los casos de uso y continúan a través del trabajo de desarrollo subsiguiente —análisis, diseño, implementación y prueba—, que termina convirtiendo en código ejecutable los casos de uso que se desarrollaban en la iteración. Por supuesto, un incremento no necesariamente es aditivo. Especialmente en las primeras fases del ciclo de vida, los desarrolladores pueden tener que reemplazar un diseño superficial por uno más detallado o sofisticado. En fases posteriores, los incrementos son típicamente aditivos.

En cada iteración, los desarrolladores identifican y especifican los casos de uso relevantes, crean un diseño utilizando la arquitectura seleccionada como guía, implementan el diseño mediante componentes, y verifican que los componentes satisfacen los casos de uso. Si una iteración cumple con sus objetivos —como suele suceder— el desarrollo continúa con la siguiente iteración. Cuando una iteración no cumple sus objetivos, los desarrolladores deben revisar sus decisiones previas y probar con un nuevo enfoque.

Para alcanzar el mayor grado de economía en el desarrollo, un equipo de proyecto intentará seleccionar sólo las iteraciones requeridas para lograr el objetivo del proyecto. Intentará secuenciar las iteraciones en un orden lógico. Un proyecto con éxito se ejecutará de una forma directa, sólo con pequeñas desviaciones del curso que los desarrolladores planificaron inicialmente. Por supuesto, en la medida en que se añadan iteraciones o se altere el orden de las mismas por problemas inesperados, el proceso de desarrollo consumirá más esfuerzo y tiempo. Uno de los objetivos de la reducción del riesgo es minimizar los problemas inesperados.

Son muchos los beneficios de un proceso iterativo controlado:

- La iteración controlada reduce el coste del riesgo a los costes de un solo incremento. Si los desarrolladores tienen que repetir la iteración, la organización sólo pierde el esfuerzo mal empleado de la iteración, no el valor del producto entero.
- La iteración controlada reduce el riesgo de no sacar al mercado el producto en el calendario previsto. Mediante la identificación de riesgos en fases tempranas del desarrollo, el tiempo que se gasta en resolverlos se emplea al principio de la planificación, cuando la gente está menos presionada por cumplir los plazos. En el método “tradicional”, en el cual los problemas complicados se revelan por primera vez en la prueba del sistema, el tiempo necesario para resolverlos normalmente es mayor que el tiempo que queda en la planificación, y casi siempre obliga a retrasar la entrega.
- La iteración controlada acelera el ritmo del esfuerzo de desarrollo en su totalidad debido a que los desarrolladores trabajan de manera más eficiente para obtener resultados claros a corto plazo, en lugar de tener un calendario largo, que se prolonga eternamente.
- La iteración controlada reconoce una realidad que a menudo se ignora —que las necesidades del usuario y sus correspondientes requisitos no pueden definirse completamente al

principio. Típicamente, se refinan en iteraciones sucesivas. Esta forma de operar hace más fácil la adaptación a los requisitos cambiantes.

Estos conceptos —los de desarrollo dirigido por los casos de uso, centrado en la arquitectura, iterativo e incremental— son de igual importancia. La arquitectura proporciona la estructura sobre la cual guiar las iteraciones, mientras que los casos de uso definen los objetivos y dirigen el trabajo de cada iteración. La eliminación de una de las tres ideas reduciría drásticamente el valor del Proceso Unificado. Es como un taburete de tres patas. Sin una de ellas, el taburete se cae.

Ahora que hemos presentado los tres conceptos clave, es el momento de echar un vistazo al proceso en su totalidad, su ciclo de vida, artefactos, flujos de trabajo, fases e iteraciones.

1.5. La vida del Proceso Unificado

El Proceso Unificado se repite a lo largo de una serie de ciclos que constituyen la vida de un sistema, como se muestra en la Figura 1.2. Cada ciclo concluye con una **versión** del producto (Apéndice C; véase también el Capítulo 5) para los clientes.

Cada ciclo consta de cuatro fases: inicio¹, elaboración, construcción y transición. Cada fase (Apéndice C) se subdivide a su vez en iteraciones, como se ha dicho anteriormente. Véase la Figura 1.3.

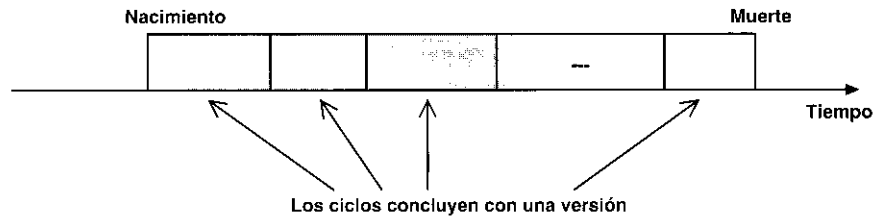


Figura 1.2. La vida de un proceso consta de ciclos desde su nacimiento hasta su muerte.

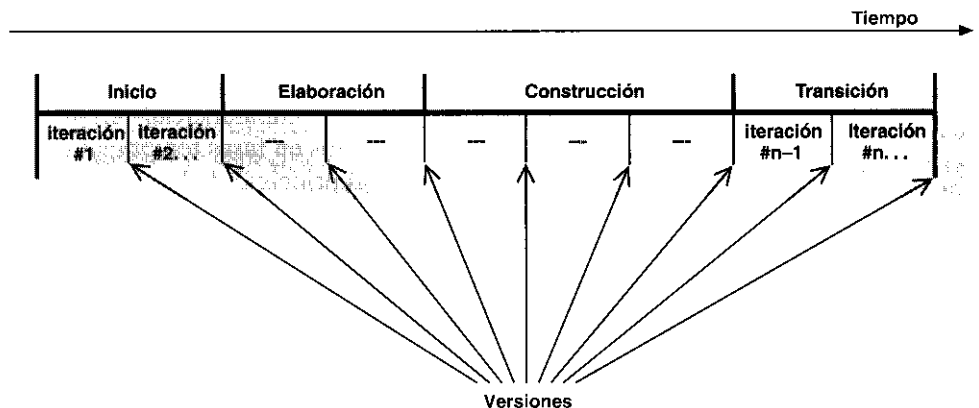


Figura 1.3. Un ciclo con sus fases e iteraciones.

¹ N. del T. También se suele utilizar el término “fase de comienzo”.

1.5.1. El producto

Cada ciclo produce una nueva versión del sistema, y cada versión es un producto preparado para su entrega. Consta de un cuerpo de código fuente incluido en componentes que puede compilarse y ejecutarse, además de manuales y otros productos asociados. Sin embargo, el producto terminado no sólo debe ajustarse a las necesidades de los usuarios, sino también a las de todos los interesados, es decir, toda la gente que trabajará con el producto. El producto software debería ser algo más que el código máquina que se ejecuta.

El producto terminado incluye los requisitos, casos de uso, especificaciones no funcionales y casos de prueba. Incluye el modelo de la arquitectura y el modelo visual —artefactos modelados con el Lenguaje Unificado de Modelado. De hecho, incluye todos los elementos que hemos mencionado en este capítulo, debido a que son esos elementos los que permiten a los interesados— clientes, usuarios, analistas, diseñadores, programadores, ingenieros de prueba, y directores —especificar, diseñar, implementar, probar y utilizar un sistema. Es más, son esos elementos los que permiten a los usuarios utilizar y modificar el sistema de generación en generación.

Aunque los componentes ejecutables sean los artefactos más importantes desde la perspectiva del usuario, no son suficientes por sí solos. Esto se debe a que el entorno cambia. Se mejoran los sistemas operativos, los sistemas de bases de datos y las máquinas que los soportan. A medida que el objetivo del sistema se comprende mejor, los propios requisitos pueden cambiar. De hecho, el que los requisitos cambien es una de las constantes del desarrollo de software. Al final, los desarrolladores deben afrontar un nuevo ciclo, y los directores deben financiarlo. Para llevar a cabo el siguiente ciclo de manera eficiente, los desarrolladores necesitan todas las representaciones del producto software (Figura 1.4):

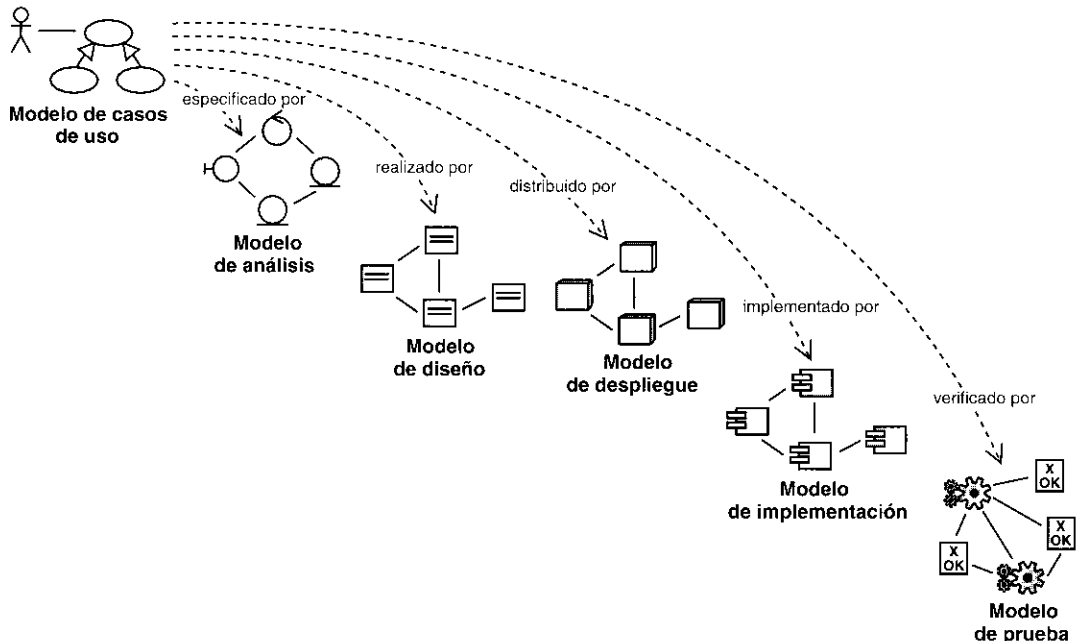


Figura 1.4. Modelo del Proceso Unificado. Existen dependencias entre muchos de los modelos. Como ejemplo, se indican las dependencias entre el modelo de casos de uso y los demás modelos.

- Un modelo de casos de uso, con todos los casos de uso y su relación con los usuarios.
- Un modelo de análisis, con dos propósitos: refinar los casos de uso con más detalle y establecer la asignación inicial de funcionalidad del sistema a un conjunto de objetos que proporcionan el comportamiento.
- Un modelo de diseño que define: (a) la estructura estática del sistema en la forma de subsistemas, clases e interfaces y (b) los casos de uso reflejados como **colaboraciones** (Apéndice A; véase también la Sección 3.1) entre los subsistemas, clases, e interfaces.
- Un modelo de implementación, que incluye componentes (que representan al código fuente) y la correspondencia de las clases con los componentes.
- Un modelo de despliegue², que define los nodos físicos (ordenadores) y la correspondencia de los componentes con esos nodos.
- Un modelo de prueba, que describe los casos de prueba que verifican los casos de uso.
- Y, por supuesto, una representación de la arquitectura.

El sistema también debe tener un modelo del dominio o modelo del negocio que describa el contexto del negocio en el que se halla el sistema.

Todos estos modelos están relacionados. Juntos, representan al sistema como un todo. Los elementos de un modelo poseen dependencias de **traza** (Apéndice A; véase también la Sección 2.3.7) hacia atrás y hacia adelante, mediante enlaces hacia otros modelos. Por ejemplo, podemos hacer el seguimiento de un caso de uso (en el modelo de casos de uso) hasta una realización de caso de uso (en el modelo de diseño) y hasta un caso de prueba (en el modelo de prueba). La trazabilidad facilita la comprensión y el cambio.

1.5.2. Fases dentro de un ciclo

Cada ciclo se desarrolla a lo largo del tiempo. Este tiempo, a su vez, se divide en cuatro fases, como se muestra en la Figura 1.5. A través de una secuencia de modelos, los implicados visualizan lo que está sucediendo en esas fases. Dentro de cada fase, los directores o los desarrolladores pueden descomponer adicionalmente el trabajo —en iteraciones con sus incrementos resultantes. Cada fase termina con un **hito** (Apéndice C; véase también Capítulo 5). Cada hito se determina por la disponibilidad de un conjunto de artefactos; es decir, ciertos modelos o documentos han sido desarrollados hasta alcanzar un estado predefinido.

Los hitos tienen muchos objetivos. El más crítico es que los directores deben tomar ciertas decisiones cruciales antes de que el trabajo pueda continuar con la siguiente fase. Los hitos también permiten a la dirección, y a los mismos desarrolladores, controlar el progreso del trabajo según pasa por esos cuatro puntos clave. Al final, se obtiene un conjunto de datos a partir del seguimiento del tiempo y esfuerzo consumido en cada fase. Estos datos son útiles en la estimación del tiempo y los recursos humanos para otros proyectos, en la asignación de los recursos durante el tiempo que dura el proyecto, y en el control del progreso contrastado con las planificaciones.

La Figura 1.5. muestra en la columna izquierda los flujos de trabajo —requisitos, análisis, diseño, implementación y prueba. Las curvas son una aproximación (no deben considerarse muy literalmente) de hasta dónde se llevan a cabo los flujos de trabajo en cada fase. Recuérdese que cada fase se divide normalmente en iteraciones o mini-proyectos. Una iteración típica pasa por los cinco flujos de trabajo, como se muestra en la Figura 1.5 para una iteración en la fase de elaboración.

² N. del T. También se suele utilizar el término “modelo de distribución”.

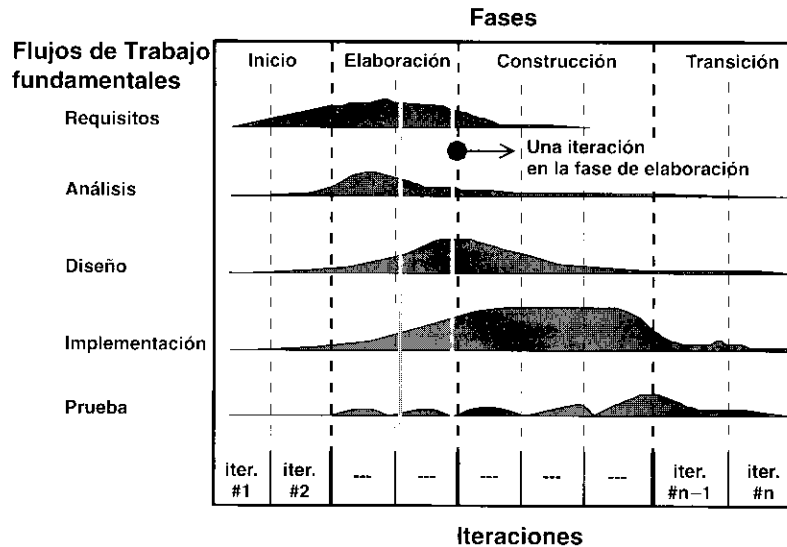


Figura 1.5. Los cinco flujos de trabajo —requisitos, análisis, diseño, implementación y prueba— tienen lugar sobre las cuatro fases: inicio, elaboración, construcción y transición.

Durante la *fase de inicio*, se desarrolla una descripción del producto final a partir de una buena idea y se presenta el análisis de negocio para el producto. Esencialmente, esta fase responde a las siguientes preguntas:

- ¿Cuáles son las principales funciones del sistema para sus usuarios más importantes?
- ¿Cómo podría ser la arquitectura del sistema?
- ¿Cuál es el plan de proyecto y cuánto costará desarrollar el producto?

La respuesta a la primera pregunta se encuentra en un modelo de casos de uso simplificado que contenga los casos de uso más críticos. Cuando lo tengamos, la arquitectura es provisional, y consiste típicamente en un simple esbozo que muestra los subsistemas más importantes. En esta fase, se identifican y priorizan los riesgos más importantes, se planifica en detalle la fase de elaboración, y se estima el proyecto de manera aproximada.

Durante la *fase de elaboración*, se especifican en detalle la mayoría de los casos de uso del producto y se diseña la arquitectura del sistema. La relación entre la arquitectura del sistema y el propio sistema es primordial. Una manera simple de expresarlo es decir que la arquitectura es análoga al esqueleto cubierto por la piel pero con muy poco músculo (el software) entre los huesos y la piel —sólo lo necesario para permitir que el esqueleto haga movimientos básicos. El sistema es el cuerpo entero con esqueleto, piel, y músculos.

Por tanto, la arquitectura se expresa en forma de vistas de todos los modelos del sistema, los cuales juntos representan al sistema entero. Esto implica que hay vistas arquitectónicas del modelo de casos de uso, del modelo de análisis, del modelo de diseño, del modelo de implementación y modelo de despliegue. La vista del modelo de implementación incluye componentes para probar que la arquitectura es ejecutable. Durante esta fase del desarrollo, se realizan los casos de uso más críticos que se identificaron en la fase de comienzo. El resultado de esta fase es una **línea base** de la arquitectura (Apéndice C; véase también Sección 4.4).

Al final de la fase de elaboración, el director de proyecto está en disposición de planificar las actividades y estimar los recursos necesarios para terminar el proyecto. Aquí la cuestión fundamental es: ¿son suficientemente estables los casos de uso, la arquitectura y el plan, y están los riesgos suficientemente controlados como para que seamos capaces de comprometernos al desarrollo entero mediante un contrato?

Durante la *fase de construcción* se crea el producto —se añaden los músculos (software terminado) al esqueleto (la arquitectura). En esta fase, la línea base de la arquitectura crece hasta convertirse en el sistema completo. La descripción evoluciona hasta convertirse en un producto preparado para ser entregado a la comunidad de usuarios. El grueso de los recursos requeridos se emplea durante esta fase del desarrollo. Sin embargo, la arquitectura del sistema es estable, aunque los desarrolladores pueden descubrir formas mejores de estructurar el sistema, ya que los arquitectos recibirán sugerencias de cambios arquitectónicos de menor importancia. Al final de esta fase, el producto contiene todos los casos de uso que la dirección y el cliente han acordado para el desarrollo de esta versión. Sin embargo, puede que no está completamente libre de defectos. Muchos de estos defectos se descubrirán y solucionarán durante la fase de transición. La pregunta decisiva es: ¿cubre el producto las necesidades de algunos usuarios de manera suficiente como para hacer una primera entrega?

La *fase de transición* cubre el periodo durante el cual el producto se convierte en versión beta. En la versión beta un número reducido de usuarios con experiencia prueba el producto e informa de defectos y deficiencias. Los desarrolladores corrigen los problemas e incorporan algunas de las mejoras sugeridas en una versión general dirigida a la totalidad de la comunidad de usuarios. La fase de transición conlleva actividades como la fabricación, formación del cliente, el proporcionar una línea de ayuda y asistencia, y la corrección de los defectos que se encuentren tras la entrega. El equipo de mantenimiento suele dividir esos defectos en dos categorías: los que tienen suficiente impacto en la operación para justificar una versión incrementada (versión *delta*) y los que pueden corregirse en la siguiente versión normal.

1.6. Un proceso integrado

El Proceso Unificado está basado en componentes. Utiliza el nuevo estándar de modelado visual, el Lenguaje Unificado de Modelado (UML), y se sostiene sobre tres ideas básicas —casos de uso, arquitectura, y desarrollo iterativo e incremental. Para hacer que estas ideas funcionen, se necesita un proceso polifacético, que tenga en cuenta ciclos, fases, flujos de trabajo, gestión del riesgo, control de calidad, gestión de proyecto y control de la configuración. El Proceso Unificado ha establecido un marco de trabajo que integra todas esas diferentes facetas. Este marco de trabajo sirve también como paraguas bajo el cual los fabricantes de herramientas y los desarrolladores pueden construir herramientas que soporten la automatización del proceso entero, de cada flujo de trabajo individualmente, de la construcción de los diferentes modelos, y de la integración del trabajo a lo largo del ciclo de vida y a través de todos los modelos.

El propósito de este libro es describir el Proceso Unificado con un énfasis particular en las facetas de ingeniería, en las tres ideas clave (casos de uso, arquitectura, y desarrollo iterativo e incremental), y en el diseño basado en componentes y el uso de UML. Describiremos las cuatro fases y los diferentes flujos de trabajo, pero no vamos a cubrir con gran detalle temas de gestión, como la planificación del proyecto, planificación de recursos, gestión de riesgos, control de configuración, recogida de métricas y control de calidad, y sólo trataremos brevemente la automatización del proceso.